

Arquitetura do Keeply MVP

1. Visão geral

O Keeply MVP será um sistema de backup em Java com agente local, backend central, painel web e armazenamento compatível com S3 via MinIO.

A proposta do MVP é validar o ciclo completo:

```
selecionar pasta local
→ gerar snapshot
→ dividir arquivos em chunks
→ calcular hashes
→ compactar chunks
→ enviar para backend
→ armazenar no MinIO
→ registrar metadados no PostgreSQL
→ restaurar arquivos via manifesto
→ validar integridade
```

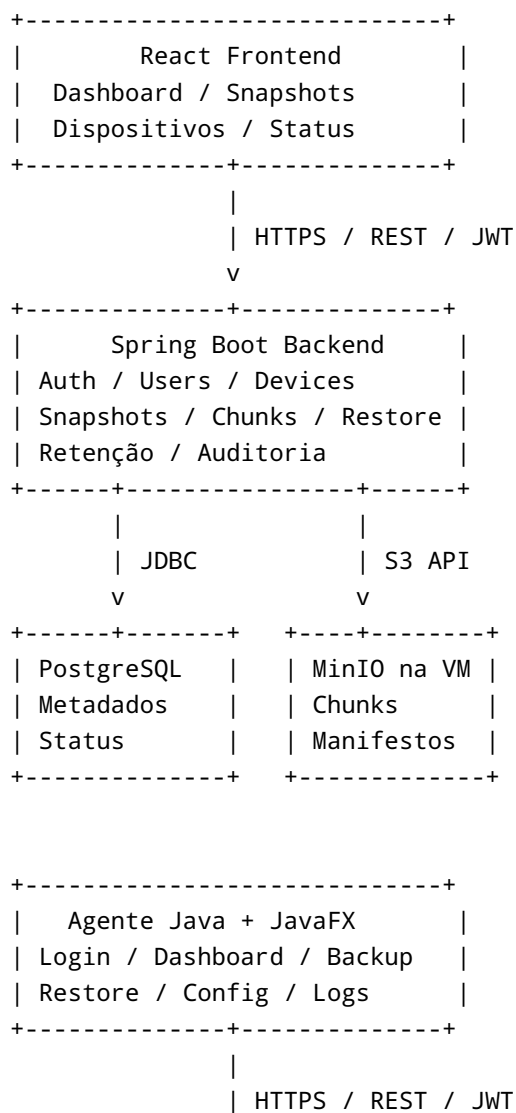
O foco principal não é apenas fazer upload. O foco é provar que um backup feito pelo agente consegue ser restaurado corretamente a partir da nuvem privada.

2. Decisões arquiteturais confirmadas

Item	Decisão
Linguagem principal	Java
Versão recomendada	Java 25 LTS, salvo exigência acadêmica por Java 21
Backend	Spring Boot
Frontend web	React
Interface local do agente	JavaFX
Banco de dados	PostgreSQL
Storage cloud do MVP	MinIO em VM
Autenticação	Email/senha + JWT
Upload	Agente envia chunks ao backend; backend grava no MinIO
Restore	Agente baixa manifesto e chunks via backend; agente reconstrói localmente
Deduplicação	Por usuário

Item	Decisão
Compressão	GZIP
Hash	SHA-256
Criptografia ponta a ponta	Fora do MVP
Retenção	Últimos 30 snapshots por usuário/dispositivo
Manifesto	JSON gerado no agente e armazenado pelo backend
Backup do MVP	Uma pasta escolhida manualmente
Visão futura	Backup completo de pastas padrão do usuário

3. Arquitetura macro



v
Spring Boot Backend

Regra central:

React não fala direto com MinIO.
Agente não fala direto com MinIO.
Somente o backend fala com MinIO.

4. Componentes principais

4.1 Agente local Java + JavaFX

Responsável por executar o backup e o restore no computador do usuário.

Funções do agente:

- autenticar no backend;
- registrar ou identificar o dispositivo;
- permitir escolher uma pasta local;
- escanear arquivos da pasta;
- gerar chunks por conteúdo;
- calcular SHA-256 dos chunks;
- compactar chunks com GZIP;
- consultar quais chunks já existem no backend;
- enviar chunks novos;
- gerar manifesto JSON;
- iniciar e finalizar snapshot no backend;
- listar snapshots disponíveis para restore;
- baixar manifesto e chunks;
- reconstruir arquivos localmente;
- validar integridade dos chunks e dos arquivos restaurados;
- exibir logs locais simples.

Telas do JavaFX:

Login
Dashboard
Backup
Restore
Configurações
Logs

Papel das telas:

Tela	Responsabilidade
Login	Autenticar o agente no backend
Dashboard	Mostrar conexão, último backup e estado geral
Backup	Escolher pasta e iniciar backup manual
Restore	Listar snapshots e restaurar para uma pasta local
Configurações	URL do backend e identificação do dispositivo
Logs	Eventos locais recentes

4.2 Backend Spring Boot

Responsável por controlar segurança, metadados, usuários, dispositivos, snapshots, chunks, retenção e acesso ao MinIO.

Funções do backend:

- autenticar usuários;
- emitir e validar JWT;
- registrar dispositivos;
- controlar permissões por usuário;
- criar snapshots com status;
- receber chunks do agente;
- verificar deduplicação por usuário;
- gravar chunks no MinIO;
- registrar metadados no PostgreSQL;
- receber manifesto JSON;
- armazenar manifesto no MinIO;
- listar snapshots para React e agente;
- entregar manifesto e chunks no restore;
- executar retenção automática dos últimos 30 snapshots;
- limpar chunks órfãos com segurança;
- registrar eventos de auditoria.

O backend é o controlador central do produto.

4.3 React Frontend

Responsável por acompanhamento visual do ambiente.

Telas mínimas:

Login
Dashboard simples
Dispositivos

O React deve responder perguntas como:

- quais dispositivos estão cadastrados;
- quando ocorreu o último backup;
- quais snapshots existem;
- quais backups falharam ou concluíram;
- quanto foi armazenado;
- qual o status geral do usuário.

O React não executa backup nem restore local.

4.4 PostgreSQL

Responsável por metadados, status, histórico e relações entre snapshots, arquivos e chunks.

O PostgreSQL não armazena arquivos grandes nem chunks binários.

Ele armazena:

- usuários;
- dispositivos;
- snapshots;
- arquivos presentes em snapshots;
- ordem dos chunks por arquivo;
- referência dos chunks no MinIO;
- restore jobs;
- auditoria.

4.5 MinIO

Responsável por armazenar objetos binários:

- chunks compactados;
- manifestos JSON.

O MinIO roda em uma VM e é acessado somente pelo backend.

5. Fluxo de backup

1. Usuário abre o agente JavaFX.
2. Usuário faz login.
3. Agente registra ou valida o dispositivo.
4. Usuário escolhe uma pasta.

5. Agente solicita ao backend a criação de um snapshot IN_PROGRESS.
6. Agente escaneia arquivos da pasta.
7. Para cada arquivo:
 - divide em chunks por conteúdo;
 - calcula SHA-256 de cada chunk original;
 - compacta o chunk com GZIP;
 - monta a lista de chunks do arquivo;
 - calcula SHA-256 final do arquivo.
8. Agente consulta no backend quais chunks já existem para o usuário.
9. Agente envia apenas chunks novos.
10. Backend salva chunks no MinIO.
11. Backend registra chunks no PostgreSQL.
12. Agente gera manifesto JSON do snapshot.
13. Agente envia manifesto ao backend.
14. Backend valida manifesto.
15. Backend salva manifesto no MinIO.
16. Backend registra arquivos e relações file_chunks no PostgreSQL.
17. Backend marca snapshot como COMPLETED.
18. Backend executa retenção automática dos últimos 30 snapshots.

Se ocorrer erro:

```
snapshot.status = FAILED  
snapshot.error_message = motivo resumido
```

6. Fluxo de restore

1. Usuário abre a tela Restore no agente JavaFX.
2. Agente consulta snapshots COMPLETED no backend.
3. Usuário escolhe um snapshot.
4. Usuário escolhe uma pasta de destino.
5. Agente solicita manifesto ao backend.
6. Backend valida usuário e dispositivo.
7. Backend busca manifesto no MinIO.
8. Backend entrega manifesto ao agente.
9. Agente lê arquivos e chunks do manifesto.
10. Para cada chunk necessário:
 - agente solicita ao backend;
 - backend busca no MinIO;
 - backend entrega ao agente;
 - agente descompacta GZIP;
 - agente calcula SHA-256 do chunk original;
 - agente compara com hash esperado.
11. Agente remonta arquivos na ordem correta.
12. Agente calcula SHA-256 final de cada arquivo restaurado.

13. Agente compara com o hash final registrado no manifesto.
14. Se tudo passar, restore é concluído.

Se qualquer validação falhar:

Restore falha com erro de integridade.
Arquivo corrompido não deve ser tratado como restaurado com sucesso.

7. Processamento dos arquivos

7.1 Chunking

O MVP usará Content-Defined Chunking simplificado.

Parâmetros definidos:

Tamanho mínimo: 512 KB
Tamanho médio: 1 MB
Tamanho máximo: 4 MB

Objetivo:

- evitar reenviar arquivo inteiro quando apenas uma parte muda;
- reaproveitar chunks já existentes do mesmo usuário;
- tornar o projeto mais próximo de um backup real.

7.2 Hash

Cada chunk terá SHA-256 calculado sobre o conteúdo original, antes da compressão.

Cada arquivo também terá SHA-256 final calculado sobre o arquivo completo original.

7.3 Compressão

A compressão será GZIP.

Ordem do backup:

```
arquivo
→ chunking
→ SHA-256 do chunk original
→ compressão GZIP
→ upload
```

Ordem do restore:

chunk compactado

- descompressão GZIP
- validação SHA-256 do chunk original
- montagem do arquivo
- validação SHA-256 final do arquivo

8. Manifesto JSON

O manifesto é gerado pelo agente e armazenado pelo backend.

Ele descreve oficialmente o conteúdo do snapshot.

Exemplo conceitual:

```
{
  "snapshotId": "uuid",
  "userId": "uuid",
  "deviceId": "uuid",
  "sourcePath": "Documents/Projeto",
  "createdAt": "2026-05-22T10:00:00Z",
  "chunking": {
    "type": "CONTENT_DEFINED",
    "minSize": 524288,
    "avgSize": 1048576,
    "maxSize": 4194304
  },
  "compression": "GZIP",
  "hashAlgorithm": "SHA-256",
  "files": [
    {
      "path": "relatorio.pdf",
      "size": 1200345,
      "lastModified": "2026-05-22T09:55:00Z",
      "sha256": "hash-final-do-arquivo",
      "chunks": [
        {
          "index": 0,
          "hash": "hash-do-chunk-1",
          "originalSize": 1048576,
          "compressedSize": 512000
        },
        {
          "index": 1,
          "hash": "hash-do-chunk-2",
          "originalSize": 151769,
          "compressedSize": 80000
        }
      ]
    }
  ]
}
```

```
    ]  
  }  
]  
}
```

Regras:

- caminhos devem ser relativos;
- não permitir `../` no restore;
- não permitir caminho absoluto no manifesto;
- o agente deve validar que o destino final fica dentro da pasta escolhida para restore.

9. Modelo lógico do banco

Tabelas confirmadas:

```
users  
devices  
snapshots  
snapshot_files  
file_chunks  
chunks  
restore_jobs  
audit_logs
```

9.1 users

```
id UUID PK  
name VARCHAR  
email VARCHAR UNIQUE  
password_hash VARCHAR  
created_at TIMESTAMP  
updated_at TIMESTAMP
```

9.2 devices

```
id UUID PK  
user_id UUID FK users.id  
name VARCHAR  
hostname VARCHAR  
os VARCHAR  
agent_version VARCHAR  
last_seen_at TIMESTAMP  
created_at TIMESTAMP
```

9.3 snapshots

```
id UUID PK
user_id UUID FK users.id
device_id UUID FK devices.id
status VARCHAR -- IN_PROGRESS, COMPLETED, FAILED
source_path TEXT
total_files BIGINT
total_original_size BIGINT
total_compressed_size BIGINT
manifest_key TEXT
started_at TIMESTAMP
completed_at TIMESTAMP
error_message TEXT
created_at TIMESTAMP
```

Regra:

Somente snapshots COMPLETED podem ser restaurados.

9.4 snapshot_files

```
id UUID PK
snapshot_id UUID FK snapshots.id
relative_path TEXT
size BIGINT
sha256 VARCHAR(64)
last_modified TIMESTAMP
created_at TIMESTAMP
```

9.5 file_chunks

```
id UUID PK
snapshot_file_id UUID FK snapshot_files.id
chunk_id UUID FK chunks.id
chunk_index INT
original_size BIGINT
compressed_size BIGINT
created_at TIMESTAMP
```

Restrição recomendada:

UNIQUE(snapshot_file_id, chunk_index)

9.6 chunks

```
id UUID PK
user_id UUID FK users.id
hash VARCHAR(64)
original_size BIGINT
compressed_size BIGINT
storage_key TEXT
created_at TIMESTAMP
```

Restrição obrigatória:

```
UNIQUE(user_id, hash)
```

Essa restrição garante deduplicação por usuário.

9.7 restore_jobs

```
id UUID PK
user_id UUID FK users.id
device_id UUID FK devices.id
snapshot_id UUID FK snapshots.id
status VARCHAR -- IN_PROGRESS, COMPLETED, FAILED
started_at TIMESTAMP
completed_at TIMESTAMP
error_message TEXT
created_at TIMESTAMP
```

9.8 audit_logs

```
id UUID PK
user_id UUID FK users.id
device_id UUID NULL FK devices.id
event_type VARCHAR
message TEXT
metadata JSONB
created_at TIMESTAMP
```

10. Layout dos objetos no MinIO

Bucket:

```
keeply
```

Estrutura:

```
keeply/  
  users/  
    {userId}/  
      chunks/  
        ab/  
          cd/  
            {sha256}.gz  
      manifests/  
        {snapshotId}.json
```

Exemplo:

```
users/123/chunks/a9/f4/a9f4...e12.gz  
users/123/manifests/550e8400-e29b-41d4-a716-446655440000.json
```

Uso de prefixos `ab/cd` evita diretórios gigantes com muitos objetos no mesmo nível.

11. APIs principais do backend

11.1 Auth

```
POST /api/auth/register  
POST /api/auth/login  
GET  /api/auth/me
```

11.2 Devices

```
POST /api/devices/register  
GET  /api/devices  
PATCH /api/devices/{deviceId}/heartbeat
```

11.3 Snapshots

```
POST /api/snapshots/start  
POST /api/snapshots/{snapshotId}/complete  
POST /api/snapshots/{snapshotId}/fail  
GET  /api/snapshots  
GET  /api/snapshots/{snapshotId}  
GET  /api/snapshots/{snapshotId}/manifest
```

11.4 Chunks

```
POST /api/chunks/check
POST /api/chunks/upload
GET /api/chunks/{hash}/download
```

`/api/chunks/check` deve aceitar lista de hashes e retornar quais já existem.

Exemplo:

```
{
  "hashes": ["abc", "def", "ghi"]
}
```

Resposta:

```
{
  "existing": ["abc"],
  "missing": ["def", "ghi"]
}
```

11.5 Restore

```
POST /api/restore-jobs
GET /api/restore-jobs
PATCH /api/restore-jobs/{restoreJobId}/complete
PATCH /api/restore-jobs/{restoreJobId}/fail
```

11.6 Dashboard React

```
GET /api/dashboard/summary
GET /api/dashboard/storage
GET /api/dashboard/recent-backups
```

12. Estados principais

12.1 Snapshot

```
IN_PROGRESS
COMPLETED
FAILED
```

12.2 Restore job

IN_PROGRESS
COMPLETED
FAILED

12.3 Device

ONLINE
OFFLINE
UNKNOWN

O estado do device pode ser calculado com base em `last_seen_at`.

13. Segurança

Regras obrigatórias:

1. Agente não possui credenciais do MinIO.
2. React não possui credenciais do MinIO.
3. Somente backend acessa PostgreSQL e MinIO.
4. Senha de usuário deve ser armazenada com hash forte.
5. JWT deve ter expiração.
6. Todo endpoint deve validar `user_id` pelo token, não pelo parâmetro recebido.
7. Restore deve bloquear path traversal.
8. Manifesto não pode conter caminho absoluto.
9. Snapshots FAILED não podem ser restaurados.
10. Chunks devem ser validados por hash no restore.

Criptografia ponta a ponta ficará fora do MVP, mas deve ser registrada como evolução futura.

Forma correta de explicar:

O MVP valida o ciclo funcional de backup e restauração com integridade.
A criptografia ponta a ponta foi planejada como evolução futura.

14. Retenção automática

Política confirmada:

Manter os últimos 30 snapshots por usuário/dispositivo.

Fluxo seguro:

1. Snapshot novo é marcado como COMPLETED.
2. Backend consulta snapshots COMPLETED do usuário/dispositivo.
3. Se houver mais de 30, seleciona os mais antigos.
4. Remove registros dos snapshots antigos.
5. Remove manifestos antigos do MinIO.
6. Identifica chunks órfãos.
7. Remove do MinIO apenas chunks que não são usados por nenhum snapshot restante.

Regra crítica:

Nunca apagar chunk somente porque um snapshot antigo foi removido.

15. Estrutura sugerida dos repositórios

Opção recomendada: monorepo

```
keeply/  
  backend/  
    build.gradle  
    src/main/java/com/keeply/backend  
    src/main/resources  
  agent/  
    build.gradle  
    src/main/java/com/keeply/agent  
  web/  
    package.json  
    src/  
  infra/  
    docker-compose.yml  
    minio/  
    postgres/  
  docs/  
    arquitetura.md  
    api.md  
    banco.md  
    demo.md
```

16. Estrutura interna do backend

```
com.keeply.backend
  auth/
    AuthController
    AuthService
    JwtService
    User
    UserRepository
  devices/
    DeviceController
    DeviceService
    Device
    DeviceRepository
  snapshots/
    SnapshotController
    SnapshotService
    Snapshot
    SnapshotFile
    FileChunk
    SnapshotRepository
  chunks/
    ChunkController
    ChunkService
    Chunk
    ChunkRepository
  storage/
    ObjectStorageService
    MinioStorageService
  restore/
    RestoreController
    RestoreService
    RestoreJob
    RestoreJobRepository
  retention/
    RetentionService
  audit/
    AuditService
    AuditLog
  config/
    SecurityConfig
    MinioConfig
    CorsConfig
```

17. Estrutura interna do agente

```
com.keeply.agent
  ui/
    LoginView
    DashboardView
    BackupView
    RestoreView
    SettingsView
    LogsView
  core/
    BackupEngine
    RestoreEngine
    FileScanner
    ManifestBuilder
  chunking/
    ContentDefinedChunker
    Chunk
  hashing/
    Sha256Hasher
  compression/
    GzipCompressor
  api/
    BackendClient
    AuthClient
    SnapshotClient
    ChunkClient
  config/
    AgentConfig
    TokenStore
  logs/
    LocalLogService
```

Regra:

A UI chama serviços do agente.
A lógica de backup não fica dentro dos controllers JavaFX.

18. Estrutura interna do React

```
src/
  pages/
    LoginPage.tsx
    DashboardPage.tsx
    DevicesPage.tsx
    SnapshotsPage.tsx
```

```
components/  
  Layout.tsx  
  StatusCard.tsx  
  SnapshotTable.tsx  
  DeviceTable.tsx  
services/  
  api.ts  
  auth.ts  
  snapshots.ts  
  devices.ts  
hooks/  
  useAuth.ts  
routes/  
  AppRoutes.tsx
```

19. Infra local para desenvolvimento

Usar Docker Compose para subir:

```
PostgreSQL  
MinIO  
Backend Spring Boot
```

O agente JavaFX e o React podem rodar fora do Docker durante desenvolvimento.

Componentes de infra:

```
PostgreSQL: 5432  
MinIO API: 9000  
MinIO Console: 9001  
Backend: 8080  
React: 5173 ou 3000
```

20. Ordem de implementação recomendada

Fase 1 — Infra e backend base

1. Criar projeto Spring Boot
2. Subir PostgreSQL e MinIO com Docker Compose
3. Configurar conexão com banco
4. Configurar MinIO client
5. Criar users e auth JWT

Fase 2 — Modelo de domínio

1. Criar tabela devices
2. Criar tabela snapshots
3. Criar tabela chunks
4. Criar snapshot_files
5. Criar file_chunks
6. Criar restore_jobs e audit_logs

Fase 3 — API de backup

1. Endpoint para iniciar snapshot
2. Endpoint para verificar chunks existentes
3. Endpoint para upload de chunk
4. Endpoint para enviar manifesto
5. Endpoint para concluir ou falhar snapshot

Fase 4 — Agente core

1. FileScanner
2. ContentDefinedChunker simplificado
3. SHA-256
4. GZIP
5. ManifestBuilder
6. BackendClient
7. BackupEngine

Fase 5 — Restore

1. Listar snapshots COMPLETED
2. Baixar manifesto
3. Baixar chunks
4. Descompactar GZIP
5. Validar hash do chunk
6. Remontar arquivo
7. Validar hash final do arquivo

Fase 6 — JavaFX

1. Login
2. Dashboard
3. Backup
4. Restore
5. Configurações
6. Logs

Fase 7 — React

1. Login
2. Dashboard
3. Lista de dispositivos
4. Lista de snapshots
5. Status dos backups

Fase 8 — Retenção e limpeza

1. Aplicar limite de 30 snapshots
2. Remover snapshots antigos
3. Remover manifestos antigos
4. Identificar chunks órfãos
5. Remover chunks órfãos do MinIO

21. O que não entra no MVP

Backup automático/agendado
Serviço rodando em segundo plano
Criptografia ponta a ponta
Deduplicação global
AWS S3 real
Backup de múltiplas pastas padrão
Painel administrativo completo
Restore em ZIP
Compartilhamento de backups
Controle avançado de permissões
Multi-tenant complexo

22. Critério de sucesso do MVP

O MVP está pronto quando for possível demonstrar:

1. Usuário faz login no agente.
2. Usuário escolhe uma pasta.
3. Agente cria snapshot.
4. Agente divide arquivos em chunks.
5. Agente envia chunks para backend.
6. Backend salva no MinIO.
7. Backend registra metadados no PostgreSQL.
8. React mostra snapshot concluído.
9. Usuário apaga a pasta local de teste.

10. Usuário abre restore no agente.
11. Agente baixa manifesto e chunks.
12. Agente restaura arquivos.
13. Sistema valida SHA-256 dos chunks e dos arquivos finais.

Se esse fluxo funcionar, o projeto está tecnicamente defensável.

23. Frase técnica para apresentação

O Keeply é um sistema de backup com agente local em Java, backend Spring Boot e armazenamento compatível com S3 via MinIO. O agente realiza chunking por conteúdo, calcula hashes SHA-256, compacta os blocos com GZIP e envia os dados para o backend. O backend centraliza autenticação, metadados, snapshots e persistência no storage. A restauração é feita a partir de manifestos JSON e chunks armazenados, com validação de integridade por chunk e por arquivo final.

24. Resumo final da arquitetura

Keeply MVP = backup restaurável com integridade.

A arquitetura correta é:

Agente JavaFX → Spring Boot → PostgreSQL + MinIO ← React Dashboard

O agente executa backup e restore.

O backend controla autenticação, metadados e storage.

O MinIO guarda chunks e manifestos.

O PostgreSQL guarda estado, histórico e relações.

O React exibe status, dispositivos e snapshots.

A prioridade técnica é provar restauração confiável, não criar uma interface bonita primeiro.